

zoh_guide

離散化ガイド：前進オイラー法

連続時間の微分方程式をコンピュータで扱える差分方程式に変換する方法を説明する。

本プログラムでは**前進オイラー法**を使用している。

1. 「離散化」とは何か

物理の方程式（熱拡散など）は**連続時間**で書かれる：

$$\frac{d\theta}{dt} = \mathbf{A}_c \theta + \mathbf{B}_c u$$

コンピュータは**離散時間**（＝ 一定時間 Δt ごとのステップ）でしか動けない。

連続時間の世界：

$\theta(t)$ —————→ 時間 t
なめらかに変化（微分方程式）

コンピュータの世界：

$\theta_0, \theta_1, \theta_2, \theta_3, \dots$ ← 飛び飛びの時刻の値（数列）
↑ Δt ↑ Δt ↑ Δt

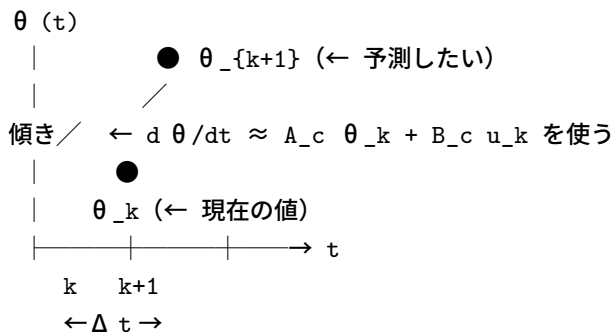
「離散化」＝ 微分方程式 → 差分方程式 への変換

2. 前進オイラー法の考え方

微分を「前進差分」で近似する

$$\left. \frac{d\theta}{dt} \right|_{t=k\Delta t} \approx \frac{\theta_{k+1} - \theta_k}{\Delta t}$$

「 Δt 秒前の傾き（＝ 現在の変化率）を使って、次の時刻の値を予測する」という考え方。



3. 離散化行列 A_d , B_d の導出

状態方程式 $\dot{\theta} = A_c \theta + B_c u$ に前進差分を代入：

$$\frac{\theta_{k+1} - \theta_k}{\Delta t} = A_c \theta_k + B_c u_k$$

両辺に Δt をかけて整理：

$$\theta_{k+1} = \underbrace{(I + A_c \Delta t)}_{A_d} \theta_k + \underbrace{B_c \Delta t}_{B_d} u_k$$

$$A_d = I + A_c \Delta t, \quad B_d = B_c \Delta t$$

4. コードとの対応 (thermal_model.py)

```
def _discretize(self):
    # A_d = I + A_c · Δ t
    A_d = np.eye(self.n) + self.A_c * self.dt

    # B_d = B_c · Δ t
    B_d = self.B_c * self.dt

    return A_d, B_d
```

numpy だけで実装できる。外部ライブラリは不要。

5. 安定条件

前進オイラー法には安定条件がある。

スカラー版 ($\dot{\theta} = -a\theta$, $a > 0$) の場合：

$$A_d = 1 - a\Delta t \quad \text{が安定} \iff |A_d| < 1 \iff 0 < a\Delta t < 2$$

行列版では、 $\mathbf{A}_d$ の全固有値の絶対値が 1 未満であることが必要。

本プログラムでの確認：

$$r \cdot \Delta t = \frac{\alpha}{(\Delta x)^2} \cdot \Delta t = \frac{1.2 \times 10^{-5}}{0.1^2} \times 10 = 0.012 \ll 0.5 \quad \checkmark$$

$\Delta t = 10\text{s} \cdot dx = 0.1\text{m}$ の設定では十分安定。

6. 主な離散化手法の比較

手法	A_d の計算	安定条件	ライブラリ
前進オイラー (本プログラム)	$I + A_c \Delta t$	$r \Delta t < 0.5$	numpy のみ
後退オイラー	$(I - A_c \Delta t)^{-1}$	無条件安定	numpy (逆行列)
ZOH (行列指数関数)	$e^{A_c \Delta t}$	無条件安定	scipy.linalg.expm

本プログラムの条件 ($r \Delta t = 0.012$) ではどの手法でも結果はほぼ同じになる。

前進オイラーはコードが最もシンプルのため採用している。